NumPy

Complete Revision Notes

Kabir Roy · 23BCE10815

VIT Bhopal University · B.Tech CSE

Foundations of Data Science

Topics: Speed & Memory · Array Creation · Indexing & Slicing · Multidimensional Arrays · Broadcasting · Data Types · Built-in Methods · Data Science Extras

0 1

1 1

1.1 1Speed Comparison

NumPy arrays are **much faster** than Python lists because they use compiled C code internally.

```
import numpy as np
import time

size = 1_000_000

# Python Lists
l1 = list(range(size))
l2 = list(range(size))

start = time.time()
add = [x + y for x, y in zip(l1, l2)]    # Loop-based
end = time.time()
print("Python Lists take", end - start)    # ~0.175 sec

# NumPy Arrays
l1 = np.array(range(size))
l2 = np.array(range(size))

start = time.time()
add = l1 + l2                              # Vectorized
end = time.time()
print("NumPy Arrays take", end - start)    # ~0.011 sec
```

NumPy was **15x faster** here. The gap grows with larger data.

1.2 1Memory Efficiency

```
import sys
import numpy as np

list_data  = list(range(1000))
numpy_data = np.array(list_data)

print("Python list size:", sys.getsizeof(list_data) *
    len(list_data), "bytes")
# ~8,056,000 bytes
```

```

print("NumPy array size:", numpy_data.nbytes, "bytes")
# 8,000 bytes  (1000 elements * 8 bytes each)
# nbytes = Number of elements x bytes per element
```

Why so different? Python lists store *pointers* to objects (8 bytes each) plus each object's overhead. NumPy stores raw numbers in a *contiguous* block — no overhead.

1.3 1Vectorization — No More Loops!

```
# Python List way (slow)
list1 = [1, 2, 3]
list_squares = [x**2 for x in list1]
print(list_squares)    # [1, 4, 9]

# NumPy way (fast + clean)
arr1 = np.array([1, 2, 3])
numpy_squares = arr1 ** 2
print(numpy_squares)   # [1 4 9]
```

Vectorization uses **SIMD** (Single Instruction, Multiple Data) — the CPU processes multiple elements simultaneously in compiled C code.

1.4 1zip() — Quick Reminder

```
l1 = [1, 2, 4]
l2 = [6, 7, 8]
list(zip(l1, l2))    # [(1, 6), (2, 7), (4, 8)]
# zip() pairs elements by position as tuples
```

2 1

2.1 1From a Python List

```
arr1 = np.array([1, 2, 3])                # 1D array
arr2 = np.array([[1,2,3],[3,4,5],[6,6,7]])  # 2D array (nested list)

# IMPORTANT: Array should ideally contain only numbers.
# Mixing types causes NumPy to upcast everything:
np.array([1, 2, 3, 4, 5, "Kabir"])
# Output: array(['1','2','3','4','5','Kabir'], dtype='<U21')  <- all become strings!
```

`np.array()` takes **only one argument** (a list or nested list). You cannot pass multiple separate lists like `np.array([1,2], [3,4])`.

2.2 1Built-in Array Creators

```
np.zeros((3, 3))        # 3x3 array filled with 0.0
np.ones((3, 5))         # 3x5 array filled with 1.0
np.full((4, 6), 5)      # 4x6 array filled with 5  -- (shape, value)
np.eye(5)               # 5x5 Identity matrix (1s on diagonal, 0s elsewhere)

np.arange(1, 100, 9)    # Like range(): start=1, stop=100, step=9
                        # Output: [1, 10, 19, 28, ... 91]

np.linspace(1, 100, 5) # 5 equally spaced values from 1 to 100 (inclusive)
                        # Output: [1., 25.75, 50.5, 75.25, 100.]
```

arange vs linspace:

- `arange(start, stop, step)` — specify the *step size*
- `linspace(start, stop, n)` — specify the *number of points*

2.3 1Array Properties

```
myarr = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12]])

print(type(myarr))    # <class 'numpy.ndarray'>
myarr.shape           # (4, 3)  -- rows x columns
myarr.size            # 12      -- total elements
myarr.ndim            # 2       -- number of dimensions
myarr.dtype           # dtype('int64')  -- default int type
myarr.nbytes          # total bytes used in memory
```

2.4 1Changing Data Type — astype()

```
myarr = np.array([[1,2,3],[4,5,6]], dtype='float32')    # set at creation
n = myarr.astype('float64')                              # convert later
print(n.dtype)    # dtype('float64')
```

2.5 1Reshape & Flatten

```
myarr = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12]])
# shape is (4, 3) -- total 12 elements

myarr.reshape((3, 4))    # New shape must have same total elements (3*4=12)
# Output: [[ 1  2  3  4], [ 5  6  7  8], [ 9 10 11 12]]

myarr.flatten()          # Convert to 1D array
# Output: [ 1  2  3  4  5  6  7  8  9 10 11 12]
```

reshape() creates a *view* (shared memory). **flatten()** creates a *copy*.

3 1

3.1 1Basic Indexing & Slicing (1D)

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9],[11,12,13]])
flat = arr.flatten()    # [1,2,3,4,5,6,7,8,9,11,12,13]

flat[0]       # 1    -- element at index 0
flat[3]       # 4    -- element at index 3
flat[3:6]     # [4, 5, 6]   -- slicing (start:stop, stop
    excluded)
flat[:5]      # [1, 2, 3, 4, 5]   -- from start to index 4
flat[3:13]    # [4, 5, 6, 7, 8, 9, 11, 12, 13]
flat[::2]     # [1, 3, 5, 7, 9, 12]   -- every 2nd element
```

3.2 1View vs Copy — CRITICAL!

```
# SLICING RETURNS A VIEW (not a copy!)
b = flat[3:7]     # b = [4, 5, 6, 7]
b[0] = 444444     # modifying b...
print(flat)       # flat is also changed!
    [1,2,3,444444,5,6,7,8,9,11,12,13]

# To avoid this, use .copy()
b = flat[3:7].copy()    # independent copy
b[0] = 22222222
print(flat)             # flat is UNCHANGED
```

Slicing = View of the same memory. If you need an independent copy, always use `.copy()`.

3.3 1Fancy Indexing

Pass a list of indices to select specific elements:

```
arr1 = np.array([10, 20, 30, 40, 50])
print(arr1[[0, 1, 2]])    # [10, 20, 30]   -- elements at
    indices 0,1,2

# Extract even numbers from [1,2,3,4,5,6]
my_arr = np.array([1, 2, 3, 4, 5, 6])
idx = [1, 3, 5]           # indices of even numbers
print(my_arr[idx])        # [2, 4, 6]
```

3.4 1Boolean Masking

```
arr2 = np.array([10, 12, 13, 344, 43])
mask = arr2 > 12          # condition creates a boolean array
print(arr2[mask])         # [13, 344, 43]  -- only elements
    where True

# One-liner:
print(arr2[arr2 > 12])    # same result
```

3.5 1Random Array Generation

```
np.random.rand(3, 3)           # floats in [0.0, 1.0)  --
    shape (3,3)
np.random.randint(1, 100, (3,3))  # integers in [1,100)  --
    shape (3,3)
np.random.random((3, 3))       # same as rand (both 0 to 1)
np.random.uniform(10, 30, (3,3))  # uniform floats in [10,
    30)
np.random.randn(3, 3)          # standard normal
    distribution (mean=0, std=1)
```

Function	Range	Use when
`rand()`	$[0, 1)$ floats	Most common, simple random data
`randint()`	[low, high) ints	Simulating discrete values
`uniform()`	[low, high) floats	Random in a custom range
`randn()`	Normal dist.	ML/statistics (Gaussian noise)

4 1

4.1 12D Indexing

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

arr[0]        # First row: [1, 2, 3]
arr[0, 1]     # Row 0, Col 1: 2  (same as arr[0][1])
arr[0:2, 1:3]  # Rows 0-1, Cols 1-2: [[2,3],[5,6]]

# Modify entire column:
arr[:, 1] = 0   # Set all elements of column 1 to 0
```

4.2 1Axis Concept

```
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

arr.sum(axis=0)    # Sum DOWN the rows (column-wise): [12, 15, 18]
arr.sum(axis=1)    # Sum ACROSS columns (row-wise):    [ 6, 15, 24]

# Same works for mean, max, min, etc.
arr.mean(axis=0)   # column means
arr.max(axis=1)    # row maximums
```

Axis Rule of thumb:

- `axis=0` → work *column-wise* (collapse rows, result has shape of columns)
- `axis=1` → work *row-wise* (collapse columns, result has shape of rows)

4.3 13D Array Indexing

```
# Think of 3D as a stack of 2D sheets
arr3D = np.array([[[1, 2, 3], [4, 5, 6]],
                  [[7, 8, 9], [10,11,12]]])

arr3D.shape           # (2, 2, 3)  -> (depth, rows, cols)
arr3D[0, 1, 2]        # Sheet 0, Row 1, Col 2 -> 6
arr3D[:, 0, :]        # All sheets, row 0, all cols: [[1,2,3],[7,8,9]]
arr3D[:, :, 0]        # All sheets, all rows, col 0: [[1,4],[7,10]]
```

5 1

Broadcasting lets NumPy perform operations on arrays of *different shapes* by automatically stretching the smaller array.

5.1 1Scalar Broadcasting

```
arr = np.array([1, 2, 3, 4, 5])
result = arr + 10          # 10 is broadcast to all elements
print(result)              # [11, 12, 13, 14, 15]

result = arr ** 2          # Vectorized squaring
print(result)              # [ 1,  4,  9, 16, 25]
```

5.2 1Array Broadcasting

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([10, 20, 30])
print(arr1 + arr2)         # [11, 22, 33]  -- element-wise, same shape

# 2D + 1D broadcasting:
arr1 = np.array([[1, 2, 3],     # shape (2, 3)
                 [4, 5, 6]])
arr2 = np.array([1, 2, 3])      # shape (3,) -- broadcast across rows

print(arr1 + arr2)
# [[2, 4, 6],
#  [5, 7, 9]]
```

5.3 1Real-World: Data Normalization

```
# Dataset: 5 samples, 3 features
data = np.array([[10, 20, 30],
                 [15, 25, 35],
                 [20, 30, 40],
                 [25, 35, 45],
                 [30, 40, 50]])

mean = data.mean(axis=0)          # Column-wise mean: [20, 30, 40]
std  = data.std(axis=0)           # Column-wise std

normalized_data = (data - mean) / std    # Broadcasting handles it all!
```

Broadcasting Rules: Arrays are compatible if, for each dimension (right-aligned), the sizes are either equal or one of them is 1.

6 1

6.1 1Integer & Float Types

```
arr = np.array([23, 4, 3, 23])
arr.dtype                          # int64 (default on 64-bit
    systems)

arr.astype("float64")              # Convert: [23., 4., 3.,
    23.]

arr = np.array([23, 4, 3, 23], dtype="float32")    # Set at
    creation

# Memory impact:
arr_int64 = np.array([1, 2, 3], dtype=np.int64)
arr_int32 = np.array([1, 2, 3], dtype=np.int32)
print(arr_int64.nbytes)    # 24 bytes  (3 * 8 bytes)
print(arr_int32.nbytes)    # 12 bytes  (3 * 4 bytes)
```

6.2 1Other Data Types

```
# String (Unicode) -- avoid in practice
arr = np.array(['apple', 'banana', 'cherry'], dtype='U10')

# Complex numbers
arr = np.array([1+2j, 3+4j, 5+6j], dtype='complex128')
# dtype='complex128': 64-bit real + 64-bit imaginary

# Object type (mixed) -- avoid, very slow
arr = np.array([{'a': 1}, [1,2,3], 'hello'], dtype=object)
```

6.3 1Data Type Quick Reference

dtype	Bytes	Range / Notes	Use case
`int8`	1	-128 to 127	Tiny integers
`int16`	2	-32768 to 32767	Small integers
`int32`	4	$\pm 2.1 \times 10^9$	General integers
`int64`	8	$\pm 9.2 \times 10^{18}$	Default int
`float32`	4	~ 7 decimal places	ML weights
`float64`	8	~ 15 decimal places	Default float
`complex128`	16	64-bit real + 64-bit imag	Signal processing
`bool`	1	True / False	Masks, flags

7 1

```
arr = np.array([3, 1, 4, 1, 5, 9, 2, 6])

np.mean(arr)              # Average value
np.std(arr)               # Standard deviation
np.var(arr)               # Variance = std^2
np.min(arr)               # Minimum value
np.max(arr)               # Maximum value
np.sum(arr)               # Sum of all elements
np.prod(arr)              # Product of all elements
np.median(arr)            # Median value

np.percentile(arr, 50) # 50th percentile (= median)
np.percentile(arr, 75) # 75th percentile (Q3)

np.argmin(arr)            # INDEX of the minimum value
np.argmax(arr)            # INDEX of the maximum value

np.unique(arr)            # Sorted unique elements
np.diff(arr)              # Differences between consecutive
   elements
np.cumsum(arr)            # Cumulative sum: [3, 4, 8, 9, 14,
   23, 25, 31]

np.log(arr)               # Natural log (ln)
np.exp(arr)               # e^x for each element

np.corrcoef(arr1, arr2)  # Correlation matrix between two
   arrays
np.linspace(0, 10, 5)    # 5 values from 0 to 10 evenly
   spaced
```

8 1

These topics weren't in your notebooks but are essential for Data Science / ML work.

8.1 1Stacking & Concatenation

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

np.vstack([a, b])        # Vertical stack (add rows)
# [[1,2],[3,4],[5,6],[7,8]]

np.hstack([a, b])        # Horizontal stack (add columns)
# [[1,2,5,6],[3,4,7,8]]

np.concatenate([a, b], axis=0)    # Same as vstack
np.concatenate([a, b], axis=1)    # Same as hstack
```

8.2 1Sorting

```
arr = np.array([3, 1, 4, 1, 5, 9])
np.sort(arr)             # Returns sorted copy: [1, 1, 3, 4,
   5, 9]
np.argsort(arr)          # Returns INDICES that would sort
   the array
                         # [1, 3, 0, 2, 4, 5]
arr.sort()               # Sorts in-place (modifies original)
```

`argsort()` is heavily used in ML — e.g., finding the K nearest neighbors by sorting distances.

8.3 1Linear Algebra (numpy.linalg)

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

np.dot(A, B)             # Matrix multiplication (dot product)
A @ B                    # Same with @ operator (Python 3.5+)

```

```
A.T                     # Transpose: rows become columns

np.linalg.det(A)        # Determinant
np.linalg.inv(A)        # Inverse of matrix
np.linalg.eig(A)        # Eigenvalues and eigenvectors
np.linalg.norm(A)       # Frobenius norm (matrix magnitude)
```

Linear algebra is the backbone of ML. Dot products are used in neural networks; eigenvectors are used in PCA (dimensionality reduction).

8.4 1where() — Conditional Selection

```
arr = np.array([10, 20, 30, 40, 50])

np.where(arr > 25, arr, 0)
# [0, 0, 30, 40, 50]  -- keep value if > 25, else replace with 0

np.where(arr > 25, 'High', 'Low')
# ['Low', 'Low', 'High', 'High', 'High']
```

8.5 1clip() — Bounding Values

```
arr = np.array([-5, 0, 3, 8, 15, 20])
np.clip(arr, 0, 10)     # All values clamped to [0, 10]
# [0, 0, 3, 8, 10, 10]
```

`clip()` is used in loss functions and gradient clipping in neural networks.

8.6 1np.nan and Handling Missing Data

```
arr = np.array([1.0, 2.0, np.nan, 4.0, np.nan])

np.isnan(arr)           # [False, False,  True, False, True]
arr[~np.isnan(arr)]     # [1., 2., 4.]  -- filter out NaNs

np.nanmean(arr)         # Mean ignoring NaN: 2.333...
```

```
np.nansum(arr)          # Sum ignoring NaN: 7.0
np.nanmax(arr)          # Max ignoring NaN: 4.0
```

Regular `np.mean()` returns `nan` if any `nan` is present. Always use `np.nanmean()`, `np.nansum()`, etc. when your data may have missing values.

8.7 1np.random with Seeds (Reproducibility)

```
np.random.seed(42)          # Old API: global seed
rng = np.random.default_rng(42)  # New API: local RNG (preferred)

rng.integers(0, 100, size=(3, 3))    # randint equivalent
rng.standard_normal((3, 3))          # randn equivalent
rng.random((3, 3))                   # rand equivalent

# Shuffling
arr = np.array([1, 2, 3, 4, 5])
np.random.shuffle(arr)     # in-place shuffle
np.random.choice(arr, size=3, replace=False)  # random sample
```

Always set a seed in ML experiments for **reproducibility** — so your random train/test split gives the same result every time.

8.8 1Splitting Arrays

```
arr = np.arange(12)
np.split(arr, 3)           # Split into 3 equal parts: [0-3], [4-7], [8-11]
np.array_split(arr, 5)     # Allows unequal splits

# For 2D arrays:
mat = np.arange(16).reshape(4, 4)
np.vsplit(mat, 2)          # Split into top-half and bottom-half
np.hsplit(mat, 2)          # Split into left-half and right-half
```

8.9 1Useful Utility Functions

```
np.abs(arr)             # Absolute value (element-wise)
np.sqrt(arr)            # Square root
np.floor(arr)           # Round down to nearest int
np.ceil(arr)            # Round up to nearest int
np.round(arr, 2)        # Round to 2 decimal places

np.zeros_like(arr)      # Array of 0s with same shape as arr
np.ones_like(arr)       # Array of 1s with same shape as arr

np.count_nonzero(arr) # Count elements that are not
    zero/False
np.any(arr > 5)         # True if ANY element satisfies
    condition
np.all(arr > 0)         # True if ALL elements satisfy
    condition
```

8.10 1Vectorized Math on Arrays

```
arr = np.array([0, np.pi/2, np.pi])

np.sin(arr)               # [0., 1., 1.2e-16]  (approx 0)
np.cos(arr)               # [1., 0., -1.]
np.tan(arr)

np.log(arr + 1)           # Natural log (add 1 to avoid
    log(0))
np.log2(arr + 1)          # Log base 2
np.log10(arr + 1)         # Log base 10

np.power(2, arr)          # 2^x for each element
np.sign(arr)              # -1, 0, or +1 for each element
```

8 1

NumPy One-Liner Reference

`np.array([1,2,3])`	Create 1D array
`np.zeros((r,c))`	r×c of zeros
`np.ones((r,c))`	r×c of ones
`np.arange(s,e,step)`	Range array
`np.linspace(s,e,n)`	n evenly spaced
`arr.reshape((r,c))`	Change shape
`arr.flatten()`	To 1D
`arr[r,c]`	2D element access
`arr[r1:r2, c1:c2]`	2D slicing
`arr[mask]`	Boolean masking
`arr.copy()`	Independent copy
`np.where(cond,a,b)`	Conditional select
`np.vstack/hstack`	Stack arrays
`np.sort / np.argsort`	Sort / sort indices
`A @ B`	Matrix multiply
`A.T`	Transpose
`np.nanmean(arr)`	Mean ignoring NaN
`np.random.seed(n)`	Reproducibility